Recommendation Systems

Big Data Analytics - Gandaki University MIT-AI

Mahesh Shakya in co. with Shiva Ram Dam
Research Associate, NAAMII
Incoming PhD Student @ Hertie AI Center for Brain Health, Uni. of Tubingen, Germany

Today

Recommendation Systems

Content-based Recommendation

- Pros/Cons

Collaborative Filtering

- User-based vs. Item-based
- Pros/Cons

Problem Statement - Recommendation System

You are part of a data science team at a commercial movie-streaming startup, “HamroMovies”. Your task is to ***build*** *and evaluate a* ***recommender system*** *that increases user watch time and satisfaction through* ***better movie suggestions****.*

Dataset Understanding

*You've been given access to the **MovieLens dataset** (small), which includes user ratings for thousands of movies. Your team must figure out **how to model the problem**, understand the data, and **determine a starting approach** to building a recommender system.*

MovieLens-small Dataset

user-id	movie-id	rating	Timestamp
1	1 (Toy Story)	4	964982703
4	21 (Money Train)	3	986935199

Movie-id	Movie Name	Genre
1	Toy Story (1995)	Adventure|Animation|Children|Comedy|Fantasy
2	Jumanji	Adventure|Children|Fantasy
3	Grumpier Old Men	Comedy|Romance

What interesting questions can we ask in the dataset to understand the problem?

- Most popular genre, movie
- Find “similar” movies?
- Find “similar” user preferences?
- Operationalize based on threshold of rating?

What interesting questions can we ask in the dataset to understand the problem?

What percent (or how many) movies has users already watched (on average) ?

Does the user preference change over time?

How do we know if a user liked the movie? (explicit vs implicit)

Baseline Solutions - What movie to recommend?

Naive / Baselines

- Trending, Highly rated, Highly viewed, highly shared

Find “similar” movies?
Find “similar” user preferences?
Operationalize based on threshold of rating?

- Use additional metadata

Baseline Solutions - What movie to recommend?

Highly rated movies? Most viewed? Trending this week/month?

Pros

- Good baseline
- Easy to implement, computational complexity

Cons

- May not reflect “real popularity”, niche group: for highly rated, Most viewed
- Not Personalized

Baseline Solutions - What movie to recommend?

Highly rated movies? Most viewed? Trending this week/month?

Pros

- Mass appeal, safe option

Cons

- Does not cater to User’s Unique preferences

How to personalize recommendations based on user preference?

Recommendation Personalized to a User

- Finding “similar” items
  - Similar Movies
    - A user rated a comedy genre movie highly. Recommend more comedy movies?

Movie features

	Genre	Year		Director	Language
ABC	Comedy, Adventure, Sci-Fi	1980- 2000	??	??	

ABC = [1, 0, 1, 4, 0.65]

DEF = [0,12,4,1,0.65]

Sim(ABC,DEF) = cosine similarity = A.B / (|A||B|)

How to convert to feature vector [number]?

Feature Encoding ABC = [0], DEF = [2] ordering

One-hot Encoding ABC = {Comedy, Adventure} ABC = [1,1,0]

Recommendation Personalized to a User

- Finding “similar” items
  - Similar Movies
    - A user rated a comedy genre movie highly. Recommend more comedy movies?

Features

	Cultural Theme	Year	Rajesh hamal	Sequel	Magne Budha	Comedy	Production House	Art Movie
Chakka panja 2	❌	2022	❌	✅	✅	✅	XYZ	❌
Kabaddi Kabaddi	✅	2023	❌	✅	❌	✅	ABC	✅

How do we operationalize this at scale?

Recommend

How do we operationalize this at scale?

For every movie highly rated by the user (K items already rated)

- Compute “similarity” with all movies in the database O(N)
- Sort by “similarity” O(N)
- Threshold and select t movies to recommend

Computation: O(K(N+N)) = O(KN)

How do we merge “recommendation” for K items already rated?

How do we operationalize this at scale?

For every movie highly rated by the user (K items already rated)

- Compute “similarity” with all movies in the database O(N)
- Sort by “similarity” O(N)
- Threshold and select t movies to recommend

Computation: O(K(N+N)) = O(KN) **Cosine similarity, Minhashing, LSH**

How do we merge “recommendation” for K items already rated?

How do we operationalize this at scale?

For every movie highly rated by the user (K items already rated)

- Compute “similarity” with all movies in the database O(N)
- Sort by “similarity” O(N)
- Threshold and select t movies to recommend

Computation: O(K(N+N)) = O(KN) **Cosine similarity, Minhashing, LSH**

How do we merge “recommendation” for K items already rated?

May be we can aggregate K movie features into a user feature vector? O(N)

Cultural Theme	Year	Rajesh hamal	Sequel	Magne Budha	Comedy	Production House	Art Movie
❌	2022	❌	✅	✅	✅	XYZ	❌

Content-based Filtering

Build item profiles: Each movie is represented by a feature vector (e.g., based on genres, actors, year, keywords).

- In MovieLens, genres like Action, Comedy, Romance can be encoded as binary features.

Build a user profile: For each user, aggregate the features of the movies they liked/rated highly.

Compute similarity: Use a similarity measure (e.g., cosine similarity) between the user’s profile and candidate movie profiles.

Recommend: Suggest movies most similar to the user's profile.

Content-based Filtering

Pros

- Personalize to user preference (history/ratings)

Cons

- Needs **rich metadata** (genre, keywords) **Only limited info may be available**
- Only looking at the current user's preference? **Can we use the available information from other user's too?**
- **Limited Discovery / Content Bubble** : "More of the same", no diversity

Collaborative Filtering

- Needs **rich metadata** (genre, keywords) **Only limited info may be available**

Can we recommend movies to users without any additional metadata available?
Can we use information available from other user's ?

User-Movie Matrix

Movie

User	Dhoom 2	Jaari	Avatar 2	…
Roshan	5.0		3.0	
Sabita		5.0	4.0	

User-Movie Matrix

Movie

User	Dhoom 2	Jaari	Avatar 2	…
Roshan	5.0		3.0	
Sabita		5.0	4.0	

Recommend ***Jaari*** to **Roshan?**

Finding “similar” users

	Dhoom 2	Jaari	Avatar 2	…
Roshan	5.0		3.0	
Sabita		5.0	4.0	

Roshan = {Dhoom2, Avaatar2}
Sabita = {Jaari, Avaatar2}

Jaccard Similarity = 2 |A ∩ B| / (|A| +|B|)
Cons

Does not take User Rating into account.

Finding “similar” users

	Dhoom 2	Jaari	Avatar 2	…
Roshan	5.0		3.0	
Sabita		5.0	4.0	

Roshan = [5,0,3,0]
Sabita = [0,5,4,0]

Cosine Similarity =A.B / |A|+|B|
Pros
Does not take User Rating into account.
Cons
Penalizes unrated movies as “negative”, Different people rate differently

Finding “similar” users

	Dhoom 2	Jaari	Avatar 2	…
Roshan	5.0		3.0	
Sabita		5.0	4.0	

Roshan = [5,0,3,0]
Sabita = [0,5,4,0]

Pearson Correlation Coefficient
S_x,y = items rated by both x and y

$$sim(x,y) = \frac{\sum_{s \in S_{xy}} (r_{xs} - \overline{r_x})(r_{ys} - \overline{r_y})}{\sqrt{\sum_{s \in S_{xy}} (r_{xs} - \overline{r_x})^2} \sqrt{\sum_{s \in S_{xy}} (r_{ys} - \overline{r_y})^2}}$$

Finding “similar” users

Jaccard Similarity = 2 |A ∩ B| / (|A| +|B|)

Does not take User Rating into account

Cosine Similarity =A.B / |A|+|B|

Penalizes unrated movies as “negative”, Different people rate differently

Pearson Correlation Coefficient

S_x,y = items rated by both x and y

$$sim(x,y) = \frac{\sum_{s \in S_{xy}} (r_{xs} - \overline{r_x})(r_{ys} - \overline{r_y})}{\sqrt{\sum_{s \in S_{xy}} (r_{xs} - \overline{r_x})^2} \sqrt{\sum_{s \in S_{xy}} (r_{ys} - \overline{r_y})^2}}$$

Recommendation as Prediction

	Dhoom 2	Jaari	Avatar 2	…
Roshan	5.0	??	3.0	
Sabita	??	5.0	4.0	

Recommendation as Prediction

From similarity metric to recommendations:

- Let r_x be the vector of user x's ratings
- Let N be the set of k users most similar to x who have rated item i
- **Prediction for item *s* of *user x*:**
  - $r_{xi} = \frac{1}{k} \sum_{y \in N} r_{yi}$
  - $r_{xi} = \frac{\sum_{y \in N} s_{xy} \cdot r_{yi}}{\sum_{y \in N} s_{xy}}$
  - Other options?
- **Many other tricks possible…**

User-based Collaborative Filtering

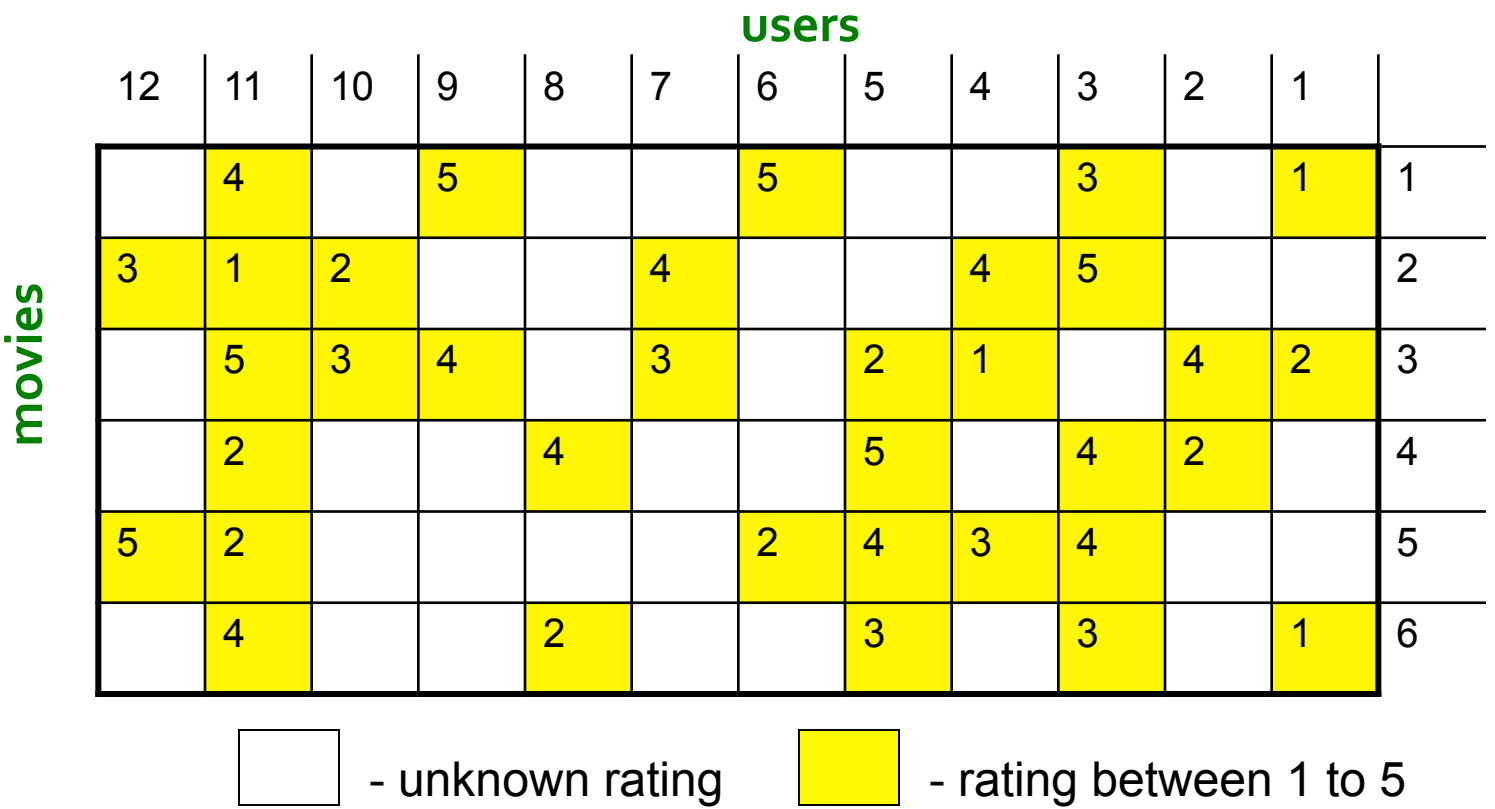
Calculate pairwise user similarity using cosine similarity.

Identify k-nearest neighbors (most similar users).

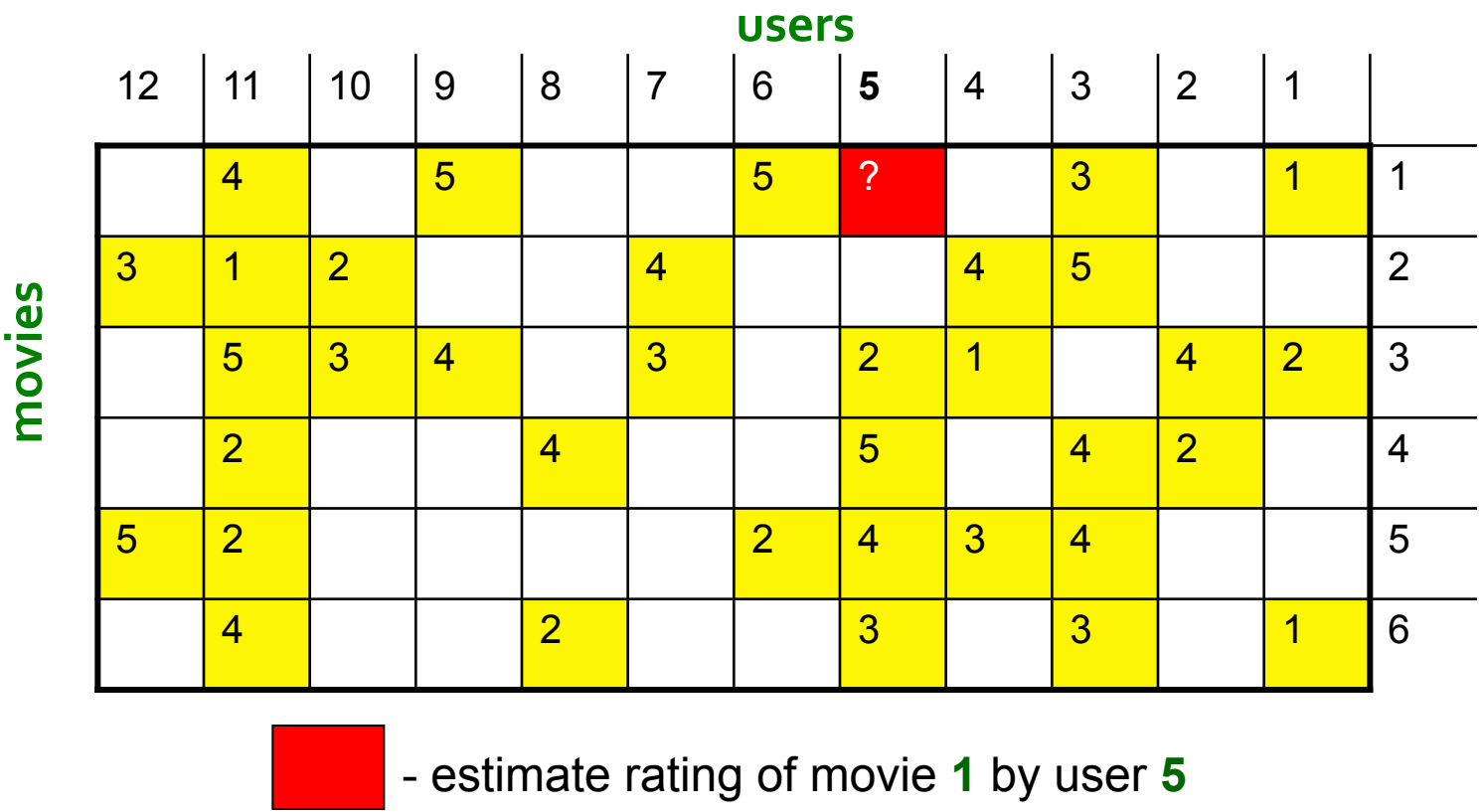
Predict scores for unrated movies using neighbors' weighted ratings.

Recommend movies with highest predicted scores.

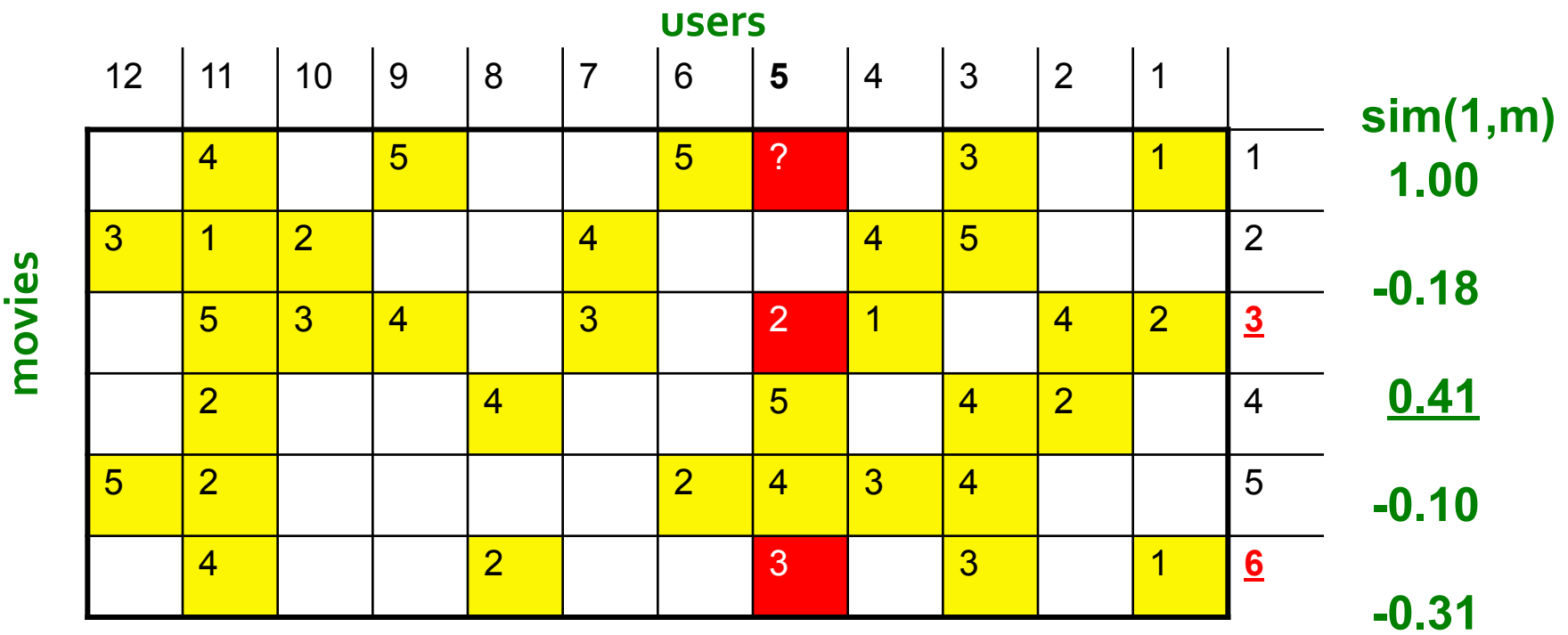
Item-Item CF (|N|=2)



Item-Item CF (|N|=2)



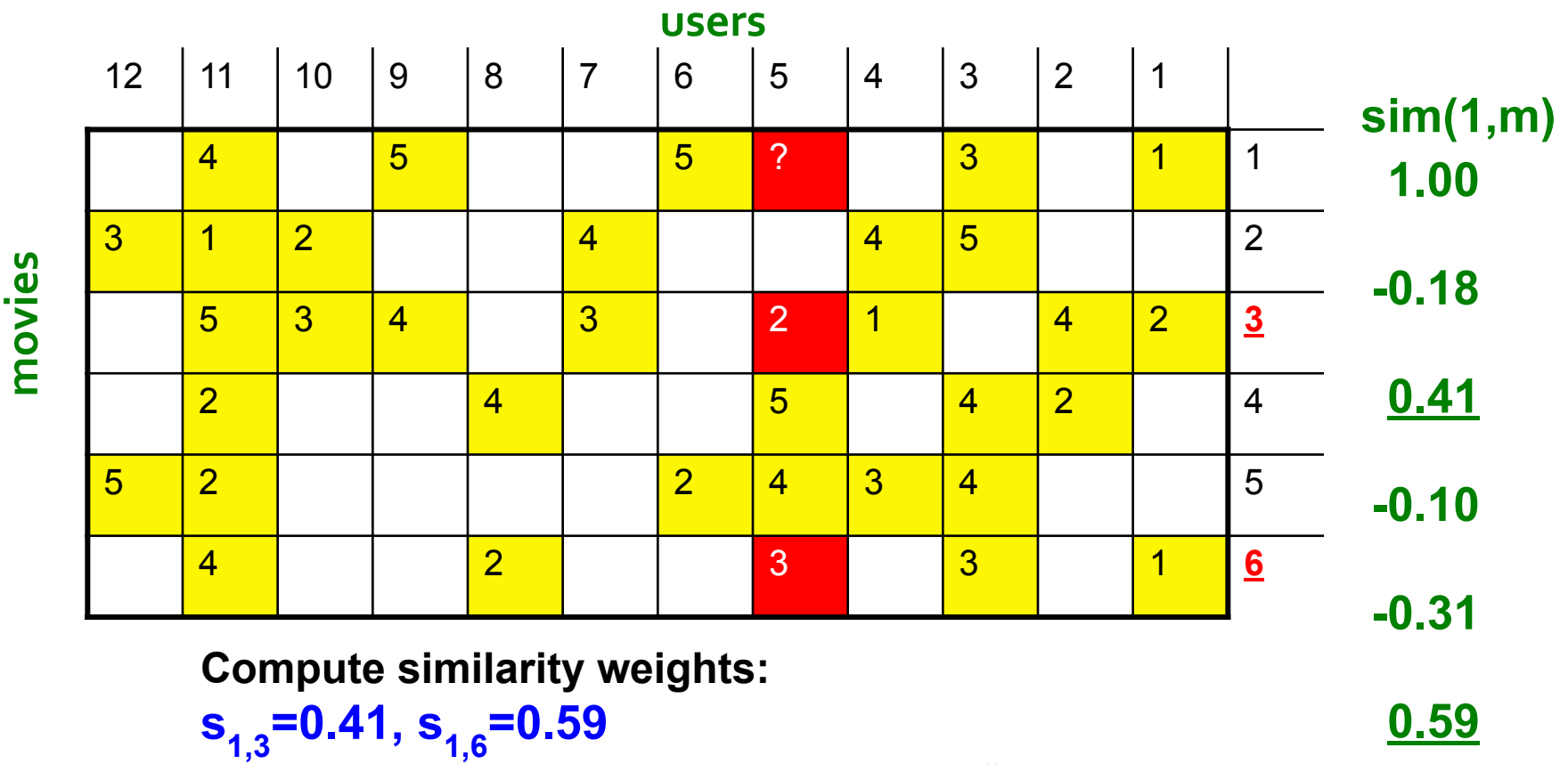
Item-Item CF (|N|=2)



Neighbor selection:
Identify movies similar to movie **1**, rated by user **5**

Here we use Pearson correlation as similarity:
1) Subtract mean rating m_i from each movie i
$m_1 = (1+3+5+5+4)/5 = 3.6$
row 1: [-2.6, 0, -0.6, 0, 0, 1.4, 0, 0, 1.4, 0, 0.4, 0]

Item-Item CF (|N|=2)



Compute similarity weights:

$s_{1,3}=0.41$, $s_{1,6}=0.59$

Item-Item CF (|N|=2)

users

movies	12	11	10	9	8	7	6	5	4	3	2	1	
		4		5			5	2.6		3		1	1
	3	1	2			4			4	5			2
		5	3	4		3		2	1		4	2	3
		2			4			5		4	2		4
	5	2					2	4	3	4			5
		4			2			3		3		1	6

Predict by taking weighted average:

$$r_{ix} = \frac{\sum_{j \in N(i;x)} s_{ij} \cdot r_{jx}}{\sum s_{ij}}$$

$r_{1.5}$ = (0.41*2 + 0.59*3) / (0.41+0.59) = 2.6

Item-based Collaborative Filtering

Calculate pairwise movie similarity using cosine similarity.

Identify k-nearest neighbors (most similar **movies**).

Predict scores for unrated movies using neighbors' weighted ratings.

Recommend movies with highest predicted scores.

Summary

Recommendation Systems

Content-based Recommendation

- Pros/Cons

Collaborative Filtering

- User-based vs. Item-based
- Pros/Cons